

AutoAdvisor AI

Lebanon & MENA Car-Buying Assistant

A 2-week solo AI engineering project brief

AutoAdvisor AI is an intelligent car-buying assistant that helps users find, compare, and evaluate cars based on budget, lifestyle, preferences, reviews, image analysis, and available dealer inventory.

Recommended market focus: Lebanon-first, MENA-ready.

The project should focus on the Lebanese used-car market for the demo because the user context, currency behavior, and local dealership flow are easy to explain. The architecture should remain regional, so the same system can later support UAE, KSA, Kuwait, Qatar, Jordan, or other MENA markets.

Primary demo currency: USD. This is practical for Lebanon because many car listings and dealership conversations are commonly priced in dollars. The system can optionally store local currencies such as AED, SAR, KWD, QAR, or LBP for future expansion.

Prepared: 2026-06-09

1. Project Summary

AutoAdvisor AI is an AutoTrader-inspired AI assistant for the used-car buying journey. The system helps users describe what they need in natural language, converts that request into structured buying preferences, searches available car inventory, analyzes car images, summarizes ratings and feedback, estimates fair price, and prepares a dealership inquiry.

The project is not a full marketplace and does not depend on unauthorized scraping of commercial websites. It uses public datasets, official vehicle APIs, curated/allowed automotive knowledge sources, seeded dealer inventory, or approved listing APIs when available.

2. Market Scope Decision

The recommended scope is Lebanon-first, MENA-ready. This gives the project a realistic local identity while keeping the design flexible for regional expansion.

Choice	Decision	Reason
Primary demo region	Lebanon	Easy to explain locally; USD pricing is natural for used cars; dealer/contact workflow is relatable.
Regional expansion	MENA / GCC compatible	Used-car platforms and buyer behavior are similar across Lebanon, UAE, KSA, Kuwait, Qatar, and Jordan.
Currency	USD primary, optional regional currencies	Keeps the demo simple while allowing AED/SAR/KWD/QAR/LBP later.
Listings	Seeded inventory + public datasets + approved APIs	Avoids unstable or unauthorized marketplace scraping.

3. Problem Statement

Buying a used car is difficult because users often do not know which cars fit their budget, lifestyle, fuel needs, maintenance expectations, and local availability. A user may ask for a city car, a family SUV, a low-maintenance luxury car, or a first car, but traditional marketplaces require the user to manually choose filters before they know what to search for.

AutoAdvisor AI solves this by acting as an intelligent advisor that understands the buyer, searches inventory, explains tradeoffs, analyzes uploaded car images, checks listing quality and safety, and helps the buyer contact a relevant dealership.

4. User Personas

- First-time buyer: wants a reliable low-maintenance car and needs guidance.
- City driver: needs a small, fuel-efficient car for traffic and parking.
- Family buyer: needs an SUV, crossover, or spacious sedan with safety and comfort.
- Luxury buyer: wants premium comfort but needs maintenance and price-risk warnings.
- Dealer/admin demo user: uploads a car listing image and expects image quality, NSFW, and attribute checks.

5. Functional Scope

Core user-facing features:

- Natural-language car recommendation chat.

- Budget, use-case, brand, fuel, transmission, body-type, and location extraction.
- Structured search over available inventory.
- RAG-based car-buying advice using curated automotive guides and review summaries.
- Car comparison between models such as Corolla vs Civic, X5 vs GLE, or Yaris vs Picanto.
- User ratings and minimal feedback summaries when review data is available.
- Fair-price estimation using an ML regression model.
- Dealer/contact workflow that prepares or stores an inquiry request.
- Vehicle image intelligence: NSFW filtering, image quality checking, car detection, color detection, body-type detection, and optional make/model estimation.

Out of scope:

- Unauthorized live scraping of AutoTrader, Dubizzle, OLX, Cars.com, CarGurus, or similar restricted marketplaces.
- Payments, financing approval, insurance quotes, loan recommendations, or legal/financial advice.
- Guaranteeing live availability unless data comes from an approved API, uploaded dealer inventory, or seeded demo data.
- Training a large computer-vision model from scratch.
- Enterprise marketplace infrastructure, advanced authentication, or multi-tenant SaaS complexity.

6. Data Strategy

The project uses a practical data strategy that supports local/regional realism without depending on risky scraping.

Data Source	Use	Scope Decision
Seeded Lebanon/MENA inventory	Demo availability, dealership matching, local examples.	Recommended primary inventory for demo reliability.
Public used-car datasets	Training price estimator and inventory search.	Use Kaggle/open CSV datasets with fields like price, year, mileage, make/model, fuel, condition.
NHTSA vPIC API	Make/model/VIN metadata validation.	Official public API for vehicle information.
Fuel economy data	MPG/fuel consumption enrichment.	Useful for city-car and running-cost recommendations.
Automotive guides/reviews	RAG knowledge base and feedback summaries.	Use permitted public pages, open datasets, or manually curated summaries.
Approved listing API	Optional live inventory/dealer data.	Only if access is available and allowed.

Marketplace examples for regional context include OLX/dubizzle Lebanon, dubizzle UAE/KSA, YallaMotor, OpenSooq, and local dealership websites. These are market references, not automatic scraping targets.

7. AI, ML, and DL Components

The project intentionally uses different model types where they make sense instead of forcing one model to solve everything.

Component	Model Type	Purpose
Intent classification	Classical ML	Routes user messages: recommendation, comparison, price check, image analysis, dealer contact, review question.
Preference extraction	LLM + schema	Extracts budget, region, brand, body type, fuel preference, use case, family size, luxury preference.
Structured inventory search	Rules + filters	Finds cars that match budget, mileage, year, body type, location, and fuel constraints.
Fair price estimator	ML regression	Predicts expected price from year, mileage, make, model, fuel, transmission, and condition.
Recommendation fit scorer	Rules + optional ML	Ranks cars by fit to user needs and constraints.
Review sentiment/topic	ML / pretrained NLP	Summarizes positives and complaints from minimal feedback/reviews.
RAG	Embeddings + LLM	Grounds buying advice in car guides, maintenance notes, review summaries, and policy/safety notes.
Vehicle image analyzer	Pretrained DL / CV	Detects vehicle presence, body type, dominant color, and optional make/model if confidence is high.
Image quality checker	OpenCV + rules	Checks blur, darkness, resolution, obstruction, and whether the car is visible.
NSFW image checker	Pretrained DL safety classifier	Rejects inappropriate uploaded images before storage or analysis.

8. Vehicle Image Intelligence

This feature handles uploaded images from users or dealers. It should use pretrained models and lightweight image-processing checks. It should not promise perfect make/model recognition.

Image workflow:

1. User uploads a car image.
2. NSFW checker runs first. Unsafe images are rejected immediately.
3. Image quality checker measures resolution, blur, brightness, and visibility.
4. Vehicle detector checks whether a car is visible.
5. Attribute extractor estimates body type, color, and optional make/model.
6. Low-confidence predictions are shown as suggestions and require user confirmation.
7. Confirmed attributes enrich the listing/recommendation workflow.

Quality checks:

- Minimum resolution threshold, for example 640x480 or higher.
- Blur score using Laplacian variance or similar OpenCV method.
- Brightness check to warn about dark images.
- Vehicle visibility check using car/object detector confidence.
- Optional image count rule for dealer listings: require at least front/side/interior images in future versions.

Output example:

```
{ "safe_image": true, "quality_status": "acceptable", "vehicle_detected": true, "body_type": "sedan",  
  "color": "white", "make_model_prediction": "Toyota Corolla", "confidence": 0.72,  
  "requires_user_confirmation": true }
```

9. System Architecture

The architecture is intentionally simple enough for one junior AI engineer while still showing production-style AI engineering decisions.

```
User / Dealer | v Streamlit Frontend | v FastAPI Backend | +--> Intent Classifier +--> Preference  
Extractor +--> Structured Car Search +--> Price Estimator +--> RAG Retriever +--> Recommendation  
Generator +--> Dealer Inquiry Workflow +--> Image Safety + Quality + Recognition Pipeline | v SQLite +  
ChromaDB + Image Storage Folder
```

10. Recommended Technology Stack

- Frontend: Streamlit for chat, recommendations, image upload, inventory search, and evaluation dashboard.
- Backend: FastAPI for clean service endpoints and testable AI workflows.
- Database: SQLite for cars, dealers, leads, reviews, evaluation results, and image metadata.
- Vector store: ChromaDB for RAG over car guides and review summaries.
- ML: scikit-learn for intent classifier, fair-price estimator, and review sentiment/topic baseline.
- DL/CV: pretrained models from Hugging Face/Ultralytics or similar for vehicle detection, image safety, and optional vehicle attributes.
- Image processing: OpenCV/Pillow for blur, brightness, resolution, dominant color, and preprocessing.
- LLM: one hosted LLM API for extraction, RAG answer generation, and natural recommendation explanations.

11. Main User Workflows

A. Recommendation workflow

User: "I need a reliable city car under \$10,000 in Lebanon." 1. Classify intent as car_recommendation. 2. Extract budget=10000, region=Lebanon, use_case=city, priorities=reliability/fuel economy. 3. Search inventory for matching cars. 4. Score cars by budget fit, mileage, age, body type, and running cost. 5. Retrieve RAG context about city cars and low-maintenance choices. 6. Generate 3-5 recommendations with tradeoffs and source-backed advice.

B. Image upload workflow

User/dealer uploads a car picture. 1. NSFW filter checks image safety. 2. Quality checker flags blur, darkness, low resolution, or missing car. 3. Vehicle recognizer estimates body type, color, and optional make/model. 4. System uses confirmed attributes to complete listing or improve recommendation.

C. Dealer connection workflow

User: "I like this Corolla. Connect me with a dealer." 1. Classify intent as dealer_contact. 2. Confirm selected car and location. 3. Match seeded/approved dealer inventory by brand, region, and availability. 4. Generate inquiry draft. 5. Store lead locally or show copy/paste message after user confirmation.

12. Database Tables

cars: id, make, model, year, price_usd, mileage_km, body_type, fuel, transmission, color, condition, region, dealer_id, availability_status

dealerships: id, name, location, phone, email, website, supported_brands, region

dealer_leads: id, created_at, selected_car_id, budget, user_location, preferred_contact_method, message_draft, status

car_reviews: id, make, model, rating, review_text, sentiment, topic, source

documents: id, title, url, category, raw_text

document_chunks: id, document_id, chunk_index, text, source_url, embedding_id

image_checks: id, image_path, nsfw_status, quality_score, blur_score, brightness_score, vehicle_detected, detected_color, detected_body_type, predicted_make_model, confidence

eval_results: id, eval_name, metric_name, metric_value, created_at

13. Evaluation Criteria

Evaluation	Metric	Target
Intent classifier	Accuracy / macro-F1	>= 80% accuracy on small labeled set
Preference extraction	Valid JSON + key-field accuracy	>= 95% valid JSON, >= 80% key fields
Price estimator	MAE / RMSE	Report baseline vs selected model
RAG retrieval	hit@3, hit@5	hit@3 >= 70%, hit@5 >= 80%
Review sentiment/topic	Accuracy or manual agreement	Report confusion/failure examples
NSFW checker	Pass/fail safety set	Reject all unsafe test images
Image quality checker	Manual agreement	Correctly flags blurry/dark/low-res examples
Vehicle image attributes	Accuracy/confidence on demo images	Use confidence threshold and manual confirmation

Recommendation quality	Manual review	8/10 scenarios should fit user needs and budget
------------------------	---------------	---

14. Two-Week Development Plan

Week 1 - data, backend, ML foundations, and first recommendation flow.

- Day 1: repo setup, FastAPI, Streamlit, SQLite, .env.example, architecture docs.
- Day 2: load/clean inventory dataset, create cars/dealers tables, build search endpoint.
- Day 3: train intent classifier and fair-price baseline model.
- Day 4: implement preference extraction with Pydantic schema and LLM prompt.
- Day 5: create RAG corpus, chunk documents, generate embeddings, implement retrieval.

Week 2 - image intelligence, dealer workflow, evaluation, and polish.

- Day 6: implement recommendation engine and fit scoring.
- Day 7: implement image pipeline: NSFW, quality checks, vehicle detection, color/type extraction.
- Day 8: build frontend pages for chat, inventory, image upload, dealership inquiry, and evaluation.
- Day 9: run evaluations for ML, RAG, extraction, image safety/quality, and recommendations.
- Day 10: finalize README, DESIGN.md, DATA.md, EVALS.md, RUNBOOK.md, and demo script.

15. Demo Scenarios

- City car: "I need a reliable city car under \$10,000 in Lebanon."
- Luxury concern: "I want something fancy like BMW or Mercedes but not expensive to maintain."
- Family SUV: "I need an SUV for my family, budget around \$18,000."
- Comparison: "Compare Toyota Corolla and Honda Civic for a first car."
- Image upload: upload a car photo; system checks NSFW, image quality, body type, color, and optional make/model.
- Dealer contact: "Connect me with a dealer for this Corolla." System prepares a safe inquiry draft.

16. Security, Safety, and Responsible Use

- Do not store unnecessary personal data; dealership inquiry should require explicit user confirmation.
- Redact phone numbers/emails from debug logs where possible.
- Reject NSFW images before further processing or storage.
- Mark availability as demo/sample unless it comes from an approved live source.
- Do not provide financing approval, legal advice, or guarantee vehicle condition.
- Warn users that used cars should be inspected by a mechanic before purchase.
- No unauthorized marketplace scraping or anti-bot bypassing.

17. Final Deliverables

- Working FastAPI backend.

- Working Streamlit frontend.
- SQLite database with inventory, dealers, reviews, documents, image checks, leads, and eval results.
- Intent classifier and fair-price estimator artifacts.
- RAG retrieval pipeline over car guides/review summaries.
- Vehicle image intelligence pipeline with NSFW and quality checks.
- Dealer inquiry workflow.
- Evaluation scripts and documented metrics.
- README.md, DESIGN.md, DATA.md, EVALS.md, RUNBOOK.md.

18. Source Notes for Project Feasibility

OLX/dubizzle Lebanon and regional platforms demonstrate the local/regional marketplace context and common listing fields such as price, mileage, fuel, location, dealer contact, and images. NHTSA vPIC provides official public vehicle metadata APIs. FuelEconomy.gov/EPA data supports fuel-economy enrichment. Public datasets such as used-car listings on Kaggle can provide a stable training/demo inventory. These sources support the project direction without requiring unauthorized scraping.

Reference examples reviewed: OLX/dubizzle Lebanon cars listings; dubizzle UAE/KSA marketplace pages; YallaMotor regional marketplace pages; OpenSooq regional listings; NHTSA vPIC API; EPA/FuelEconomy.gov vehicle data; public used-car datasets.

19. Final Approval Summary

AutoAdvisor AI is a realistic, locally relevant, and technically rich AI engineering project. It combines natural-language chat, structured search, RAG, classical ML, price estimation, review analysis, dealer inquiry generation, and computer-vision image checks. The recommended market scope is Lebanon-first and MENA-ready, with USD as the primary demo currency and regional expansion supported by design.